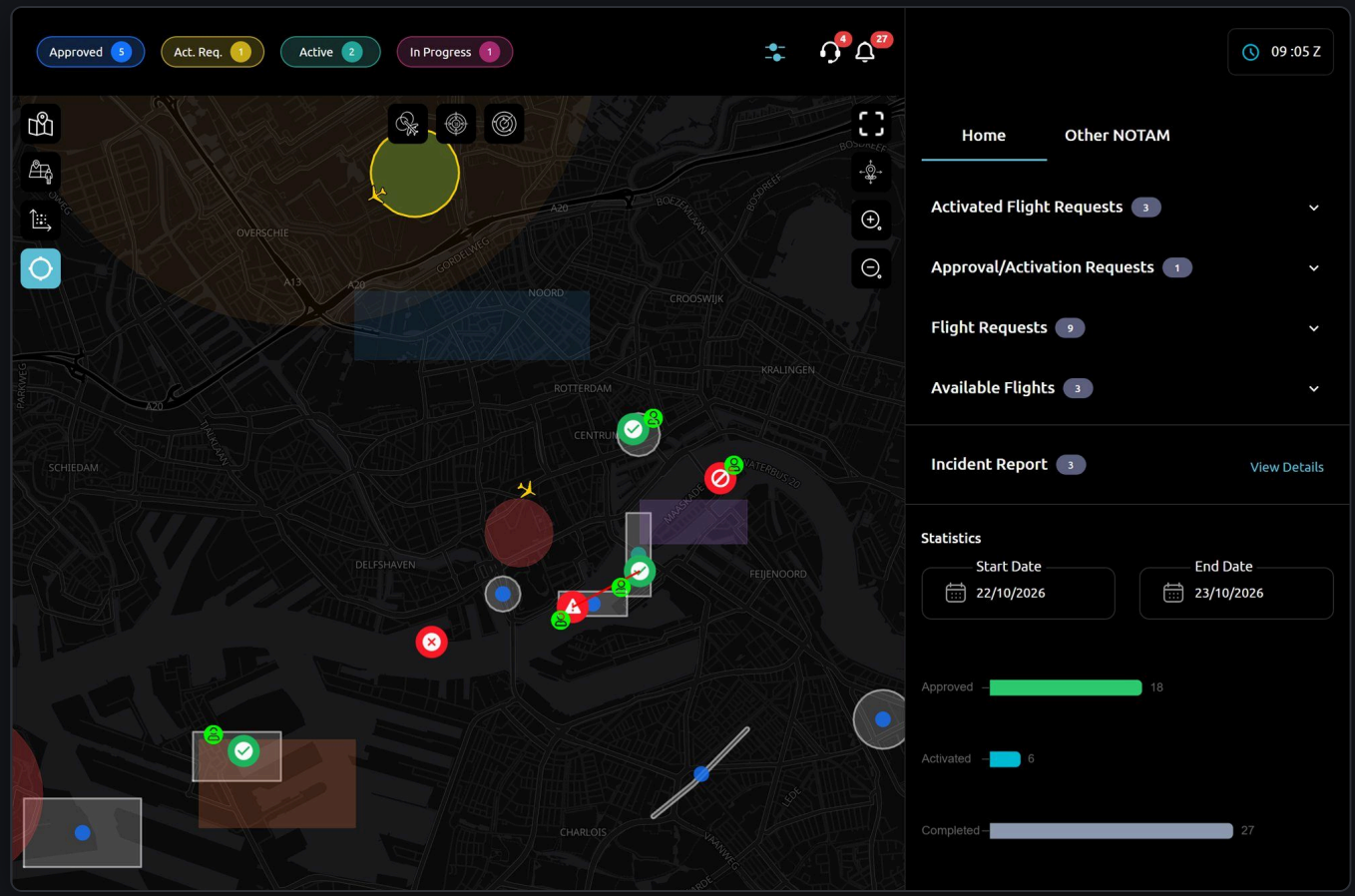
PORTFOLIO

Hamzah Hashmi

Senior Frontend Developer

React · JavaScript · TypeScript · Testing

Five years at AstraUTM, a Thales company, building the web apps that aviation authorities and drone operators use to approve flights, run the control room and keep airspace safe, for clients in the UAE and Finland.



Dubai, UAE

hamzah.hashmi92@gmail.com

+971 54 423 7454

linkedin.com/in/hamzahashmi92

All screenshots show the real interfaces filled with fictional demo data.

ABOUT ME

I build React features end to end and make them easy to test

Real-time dashboards, notifications and alerts, approval workflows and secure sign-in. I ship features with their unit tests and the test IDs QA automation relies on.

What I work on most

- Real-time features: notifications, alerts with sound, live COMMS and map layers, on TanStack Query and Zustand.
- Approval workflows for flight requests, organisations and operators, including amendments and payments.
- Testability: 330+ unit test cases, 260+ test IDs and Cypress page objects for the QA team.
- Front-end security: authenticated downloads, session-expiry handling, safe routing and dependency fixes.
- Release integration: promoting builds from development to staging and production.

SKILLS

Frontend	React, JavaScript (ES6+), TypeScript, HTML, CSS, SCSS, Bootstrap, Tailwind, Vite, webpack
Data and state	TanStack Query (infinite queries, optimistic updates), Zustand, React Context, Redux, React Hook Form, Zod
Maps	Mapbox GL JS, deck.gl, Google Maps JavaScript API, Turf.js
Testing	Vitest, Jest, React Testing Library, MSW, Cypress (page objects)
Security	OTP and national-ID sign-in, authenticated file access, SonarQube and Snyk findings
Platform	ASP.NET MVC (Razor, jQuery), i18next and Paraglide, Arabic right-to-left
Tools	Git, GitLab, GitHub, CI/CD, SonarQube, Snyk

5 years
at AstraUTM, a Thales company

208
merge requests into the current platform

330+
unit test cases written

260+
test IDs added for QA automation

WHERE THIS WORK COMES FROM

One team, four web products

AstraUTM builds drone traffic management software for aviation authorities. I worked across its web products, from the first React app to the current platform.

PRODUCT	BUILT WITH	MY WORK
AstraUTM platform	React 18, Vite, TanStack Query, Mapbox	Notifications, dashboard, approvals, tests, QA automation
SCNS, the UAE drone platform	React 16, Google Maps	Flight-request review, approvals and payments, security
Authority portal	ASP.NET MVC, jQuery, Google Maps	Two-step sign-in, live traffic alerts
Fintraffic Sky, Finland	ASP.NET MVC, Google Maps	Airspace layers on the public map
TypeScript platform (2024)	TypeScript, TanStack Router, Zod	Zone, alert and deconfliction screens
End-to-end tests	Cypress, TypeScript	Page objects and test IDs

The React apps were run from their source with fictional demo data. The ASP.NET pages were rebuilt from their original markup, styles and scripts. Client logos are removed. Code samples are short examples written for this portfolio.

CONTENTS

ASTRAUTM PLATFORM

01 A notification centre with live alerts and sound 04
02 A ready signal for the live dashboard 05
03 Test IDs and Cypress page objects for the QA team 06
04 Unit and component tests 07
05 One form to create and edit operator users 08
06 A tactical view for approving authorities 09
07 Population density on the map 10
08 Airspace conflicts while planning a flight 11
More work on the AstraUTM platform 12
09 Code reviews 13

SCNS, UAE

10 The flight-request review page 14
11 Organisation approval and payment requests 15
12 Security fixes and safe navigation 16
More work on SCNS 17

AUTHORITY PORTAL

13 Two-step sign-in for the control-room portal 18
14 Live traffic alerts in the control room 19

FINTRAFFIC SKY, FINLAND

15 Airspace layers on a public map 20

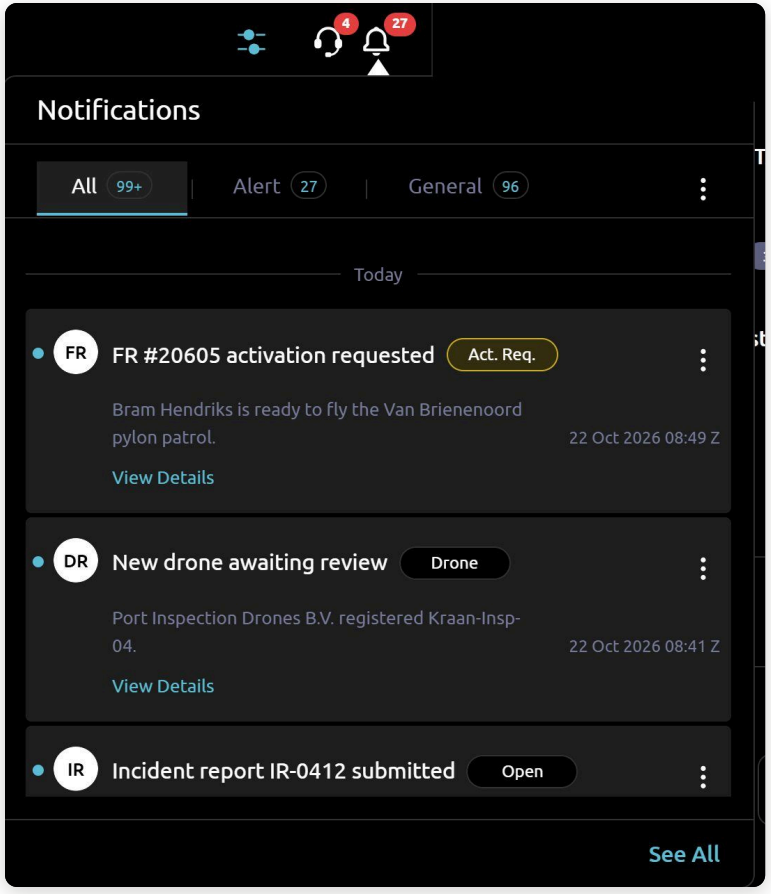
TYPESCRIPT PLATFORM

16 Building the platform in TypeScript 21

01 A notification centre with live alerts and sound

React · TanStack Query · Zustand · TypeScript · Vitest

Pilots and authority staff get every update in one place: unread counts on the bell, three tabs, links straight to the record, and a sound when something new arrives. I built the feature and later rebuilt its data layer.



The bell popup: unread badges per tab, rows for each kind of record.

```
// mark as read: update the list first,
// roll back if the server says no
const markRead = useMutation({
  mutationFn: NotificationApi.markAsRead,
  onMutate: (ids) =>
    applyOptimisticRead(client, key, ids),
  onError: (_err, _ids, ctx) =>
    rollback(client, key, ctx),
  onSettled: () =>
    client.invalidateQueries({ queryKey: key }),
});

async function applyOptimisticRead(client, key, ids) {
  const read = new Set(ids);
  await client.cancelQueries({ queryKey: key });
  const previous = client.getQueryData(key);
  client.setQueryData(key, (data) => ({
    ...data,
    pages: data.pages.map((page) => ({
      ...page,
      notifications: page.notifications.map((n) =>
        read.has(n.id) ? { ...n, isRead: true } : n),
    })),
  }));
  return { previous };
}
```

THE PROBLEM

Updates about flight requests, drones, incidents and users had no single place to land, and new ones were easy to miss.

MY PART

- The bell popup with Alert, General and All tabs, unread badges and links by record type.
- The full notifications page, grouped by day, with infinite loading.
- Bulk and single mark-as-read and delete, with a confirmation step.
- A chime when the unread count rises and a looping siren during an emergency, both behind a global mute.
- A typed notification API in TypeScript with a query-key factory, and updates that show instantly and roll back on failure.

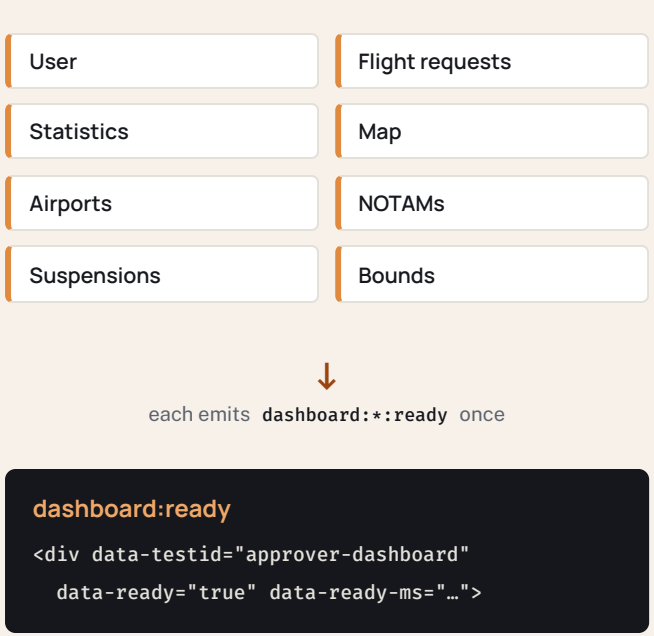
RESULT

85% of the notification code is mine, and I wrote 43 of its 44 test cases.

02 A ready signal for the live dashboard

React hooks · CustomEvent · TanStack Query

The dashboard loads eight data sources at once. I made it announce on its root element when every one of them has rendered, and how long that took, so automated tests can start at exactly the right moment.




```
// child effects run before the parent subscribes,
// so remember what already fired
const fired = new Set();

export function emitReady(name) {
  if (fired.has(name)) return;
  fired.add(name);
  window.dispatchEvent(new CustomEvent(name));
}

export function waitForAll(names, onReady) {
  const pending = new Set(
    names.filter((n) => !fired.has(n)));
  const done = () => pending.size === 0 && onReady();
  const off = [...pending].map((name) => {
    const handler = () => { pending.delete(name); done(); };
    window.addEventListener(name, handler, { once: true });
    return () => window.removeEventListener(name, handler);
  });
  done();
  return () => off.forEach((f) => f());
}

// today polls every 5 s; past days are cached for good
useQuery({
  queryKey: ["dashboard flights", start, end, isPastDay],
  refetchInterval: isPastDay ? false : 5000,
  staleTime: isPastDay ? Infinity : 0,
});
```

THE PROBLEM

One element appearing on screen doesn't mean the dashboard is ready: eight sources load independently, and tests need to know when all of them are in.

MY PART

- A ready event for each of the eight sources, emitted once per session.
- A waiter that also counts events fired before it subscribed, with full listener clean-up.
- `data-ready` and `data-ready-ms` on the dashboard root, for tests and load-time tracking.
- Flights loaded per calendar day with TanStack Query: today refreshes every 5 seconds, past days are fetched once and cached.
- 10 test cases for early, late and repeated events.

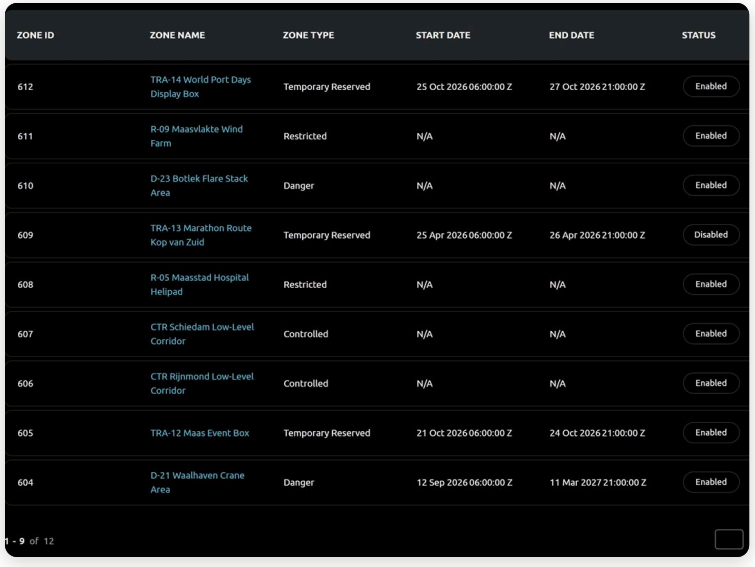
RESULT

The dashboard reports when it is ready and how long that took, and switching back to a past day loads from cache instead of the network.

03 Test IDs and Cypress page objects for the QA team

Cypress · TypeScript · data-testid

The QA team's Cypress suite needs stable hooks into the UI. I added half of the app's test IDs and wrote TypeScript page objects for the screens they cover.



ZONE ID	ZONE NAME	ZONE TYPE	START DATE	END DATE	STATUS
612	TRA-14 World Port Days Display Box	Temporary Reserved	25 Oct 2026 06:00:00 Z	27 Oct 2026 21:00:00 Z	Enabled
611	R-09 Maasvlakte Wind Farm	Restricted	N/A	N/A	Enabled
610	D-23 Botlek Flare Stack Area	Danger	N/A	N/A	Enabled
609	TRA-13 Marathon Route Kop van Zuid	Temporary Reserved	25 Apr 2026 06:00:00 Z	26 Apr 2026 21:00:00 Z	Disabled
608	R-05 Maasstad Hospital Helipad	Restricted	N/A	N/A	Enabled
607	CTR Schiedam Low-Level Corridor	Controlled	N/A	N/A	Enabled
606	CTR Rijnmond Low-Level Corridor	Controlled	N/A	N/A	Enabled
605	TRA-12 Maas Event Box	Temporary Reserved	21 Oct 2026 06:00:00 Z	24 Oct 2026 21:00:00 Z	Enabled
604	D-21 Waalhaven Crane Area	Danger	12 Sep 2026 06:00:00 Z	11 Mar 2027 21:00:00 Z	Enabled

1 - 9 of 12

The zones list. Its page object reads the grid by row, column and test ID.

```ts
// page object: the test talks to the screen,
// not to its CSS
export class AimZonesPage {
  private root = ".data-section";

  refresh(): Clickable {
    return new Element(
      `${this.root} [data-testid="refresh"]`);
  }

  list(): ZoneListTable {
    return new ZoneListTable(`${this.root} .ag-root`);
  }
}

class ZoneListTable {
  constructor(private sel: string) {}

  rowById(id: number): ZonesTableRow {
    this.rows().waitFor(toAppear);
    return new ZonesTableRow(this.sel, String(id));
  }
}

// typed values instead of magic numbers
export enum NotificationTab {
  ALL = 0, GENERAL = 1, ALERT = 2,
}
```

THE PROBLEM

Tests that found elements by visible text or CSS class broke whenever wording or styling changed.

MY PART

- 260+ distinct test IDs across the dashboard, notifications, comments, approvals and zone screens, including two clean-up merge requests covering 45 files.
- Cypress page objects in TypeScript for the zones and zone-types screens, merged into the suite.
- Page objects for the notification panel and list, with separate classes for the panel and each row and typed enums for tabs and record types.
- Review comments asking colleagues to use test IDs in their tests.

RESULT

Half of all test IDs in the app are mine: 275 of 564.

04 Unit and component tests

Vitest · React Testing Library · MSW

I write tests with every feature, using Vitest, React Testing Library and MSW, including the shared components the whole app uses. 330+ of the platform's test cases are mine.

```
// the chime plays on a rise, never on first load
it("plays only when the unread count goes up", () => {
  const { rerender } = renderHook(
    ({ count }) => useNotificationAudio({ unreadCount: count }),
    { initialProps: { count: 3 }, wrapper },
  );
  expect(play).not.toHaveBeenCalled();

  rerender({ count: 5 });
  expect(play).toHaveBeenCalledTimes(1);

  rerender({ count: 4 });
  expect(play).toHaveBeenCalledTimes(1);
});

it("ignores a blocked autoplay", async () => {
  play.mockRejectedValueOnce(
    new DOMException("blocked", "NotAllowedError"));
  const { rerender } = renderHook(...);
  rerender({ count: 9 });
  await waitFor(() =>
    expect(console.error).not.toHaveBeenCalled());
});
```

WHAT MY TESTS CHECK	CASES
Notification sound, badges, tabs and rows	43
Operator user create, edit, upload, block	21
COMMS alerts, including session expiry	17
Date ranges in UTC and local time	16
Overlapping flights: load more, error, retry	15
Commercial organisation editing	13
Dashboard ready events	10
Shared components: grid cells, tooltips, file input, visibility wrapper	80+

330+
test cases written

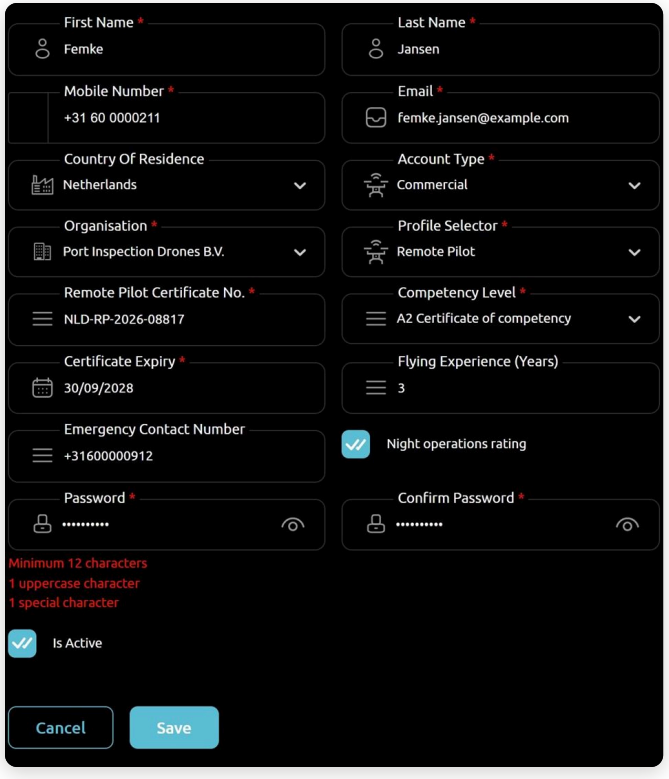
82
test files I created

8,200+
lines of test code

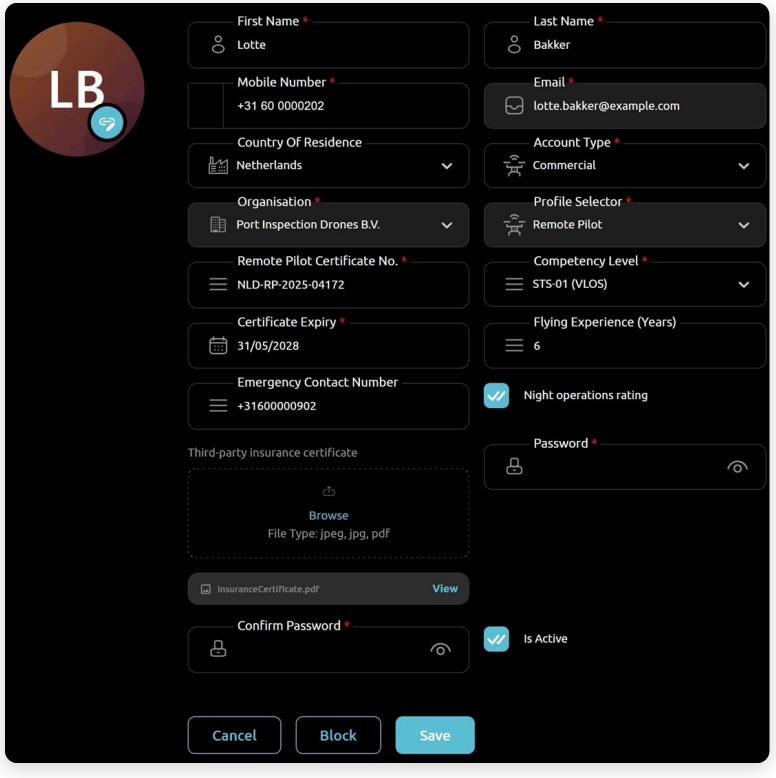
05 One form to create and edit operator users

React · TanStack Query · Zod

Every authority defines its own extra fields for operator users. I rebuilt the create and edit pages as one template-driven form that replaced three older components.



Adding a user: the authority's custom fields and the password rules.



Editing a user, with the profile picture and the super-admin block action.

THE PROBLEM

Three separate components handled creating, editing and profile settings, each with its own copy of the form logic.

MY PART

- One page that renders the authority's template fields alongside the standard ones, loaded with `useSuspenseQuery`.
- Password rules as a Zod schema, with errors shown on each field.
- Separate mutations for saving the user and uploading the profile picture.
- Block and unblock for super-admins.
- 21 test cases for create, update, upload, validation and blocking.

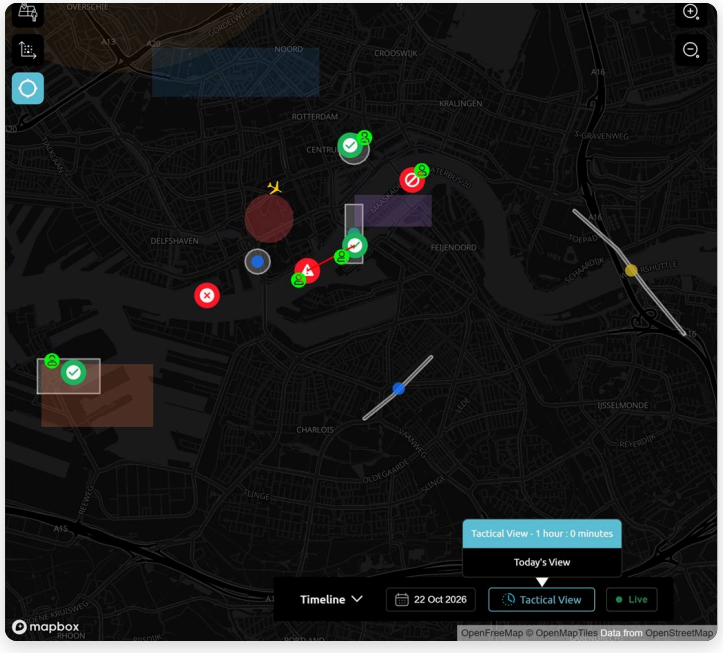
RESULT

One page, one data flow and one test file now replace three components and 1,100+ lines.

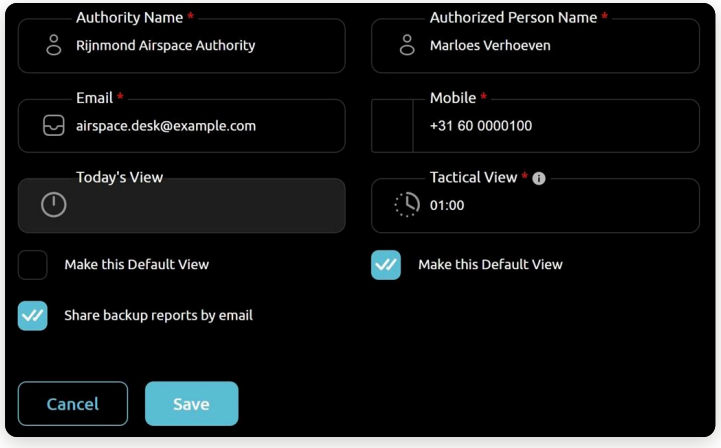
06 A tactical view for approving authorities

React · Zustand · Floating UI

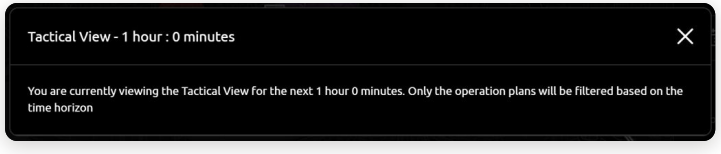
Authorities that approve flights close to take-off care about the next hour, not the whole day. I added a default view per authority and a tactical window that filters the dashboard.



The dashboard in tactical view, with the view switch open.



Each authority sets its default view and time window.



The message approvers see on their first visit.

THE PROBLEM

Every approver saw the full day of flight requests, even when only the next hour needed a decision.

MY PART

- A default view (Today or Tactical) and a time window in each authority's settings.
- A first-visit message that explains the view, and a floating switch on the dashboard.
- Flight requests filtered to the tactical window.
- A Zustand store with a typed view enum, shared by the switch and the filters.

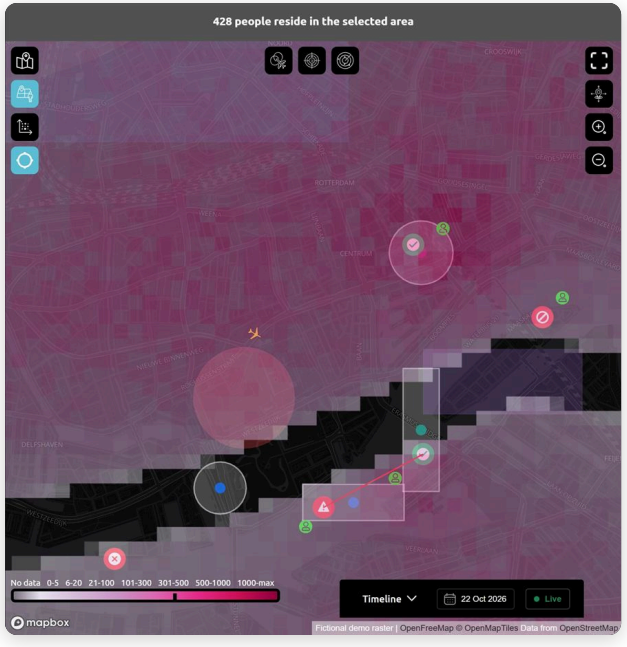
RESULT

Approvers open the dashboard on the view their authority chose, focused on the flights that need them now.

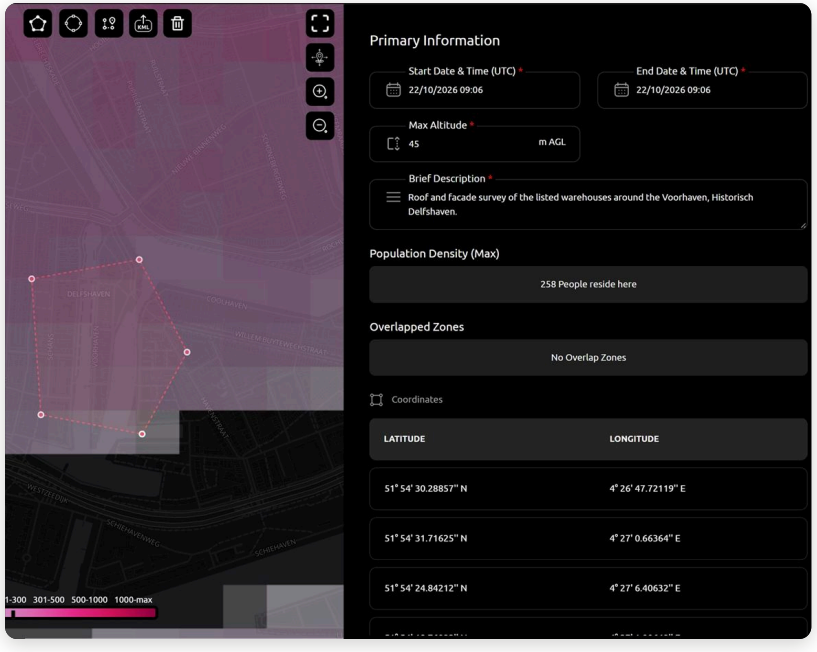
07 Population density on the map

Mapbox GL · TanStack Query · Zustand

A flight over people carries more risk. I added a population layer to the map and an estimate of how many people live under a point or a planned flight area.



The population layer, its legend and the estimate for a clicked point.



The estimate for a drawn flight area while planning.

THE PROBLEM

Approvers and pilots had no quick way to see how populated an area was before a flight.

MY PART

- A raster population layer with a toggle and a legend.
- A click on the map shows how many people live within 100 m.
- A banner with the estimate for the area drawn in a new flight request.
- A hook that fetches the estimate and keeps the result in a shared Zustand store for the map and banner.

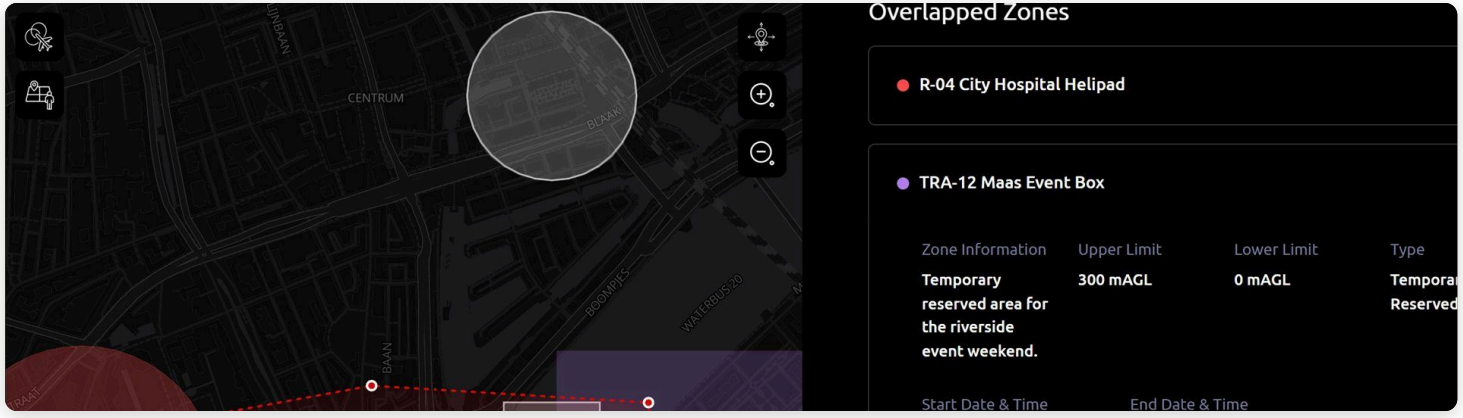
RESULT

Population is visible on the map and in every new flight request, before anyone approves it.

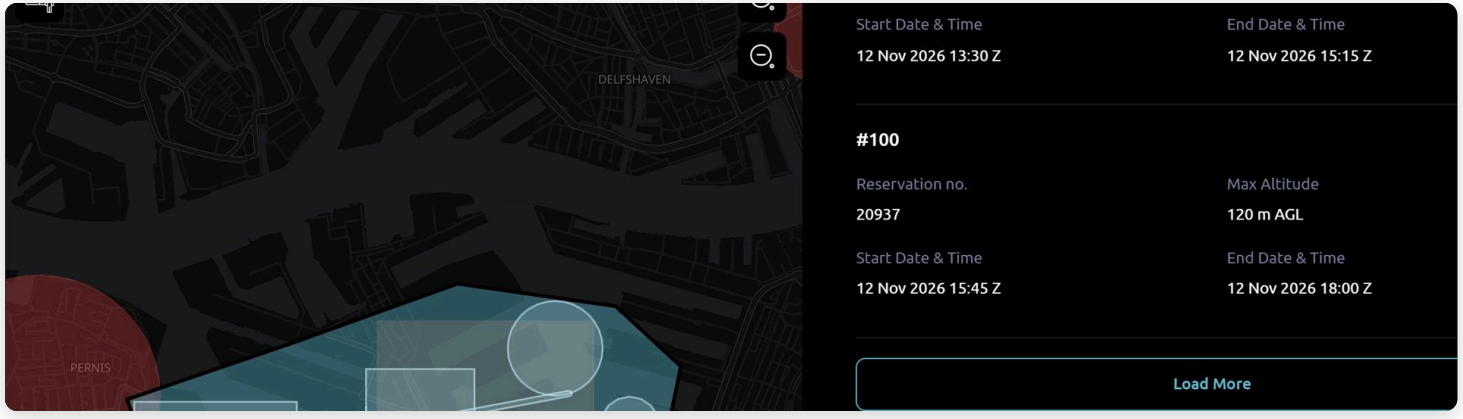
08 Airspace conflicts while planning a flight

React · Infinite queries · Mapbox GL

While a pilot draws a flight area, the form shows which zones it crosses and which approved flights it overlaps, before the request is sent.



Zones the drawn area crosses, with limits, type and validity.



Overlapping flights, loaded page by page.

THE PROBLEM

Pilots found out about restricted zones and conflicting flights only after the authority reviewed their request.

MY PART

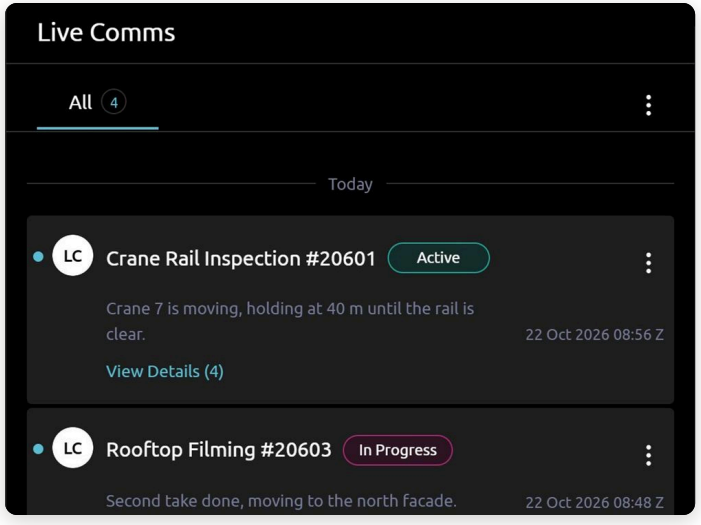
- An accordion of overlapped zones: limits, type, validity in UTC or local time, and terms.
- A paginated list of overlapping flights on an infinite query, with loading, error and retry states.
- The approver layer on the map, and a fix so overlaps update in real time.
- 15 of the 17 test cases for these components.

RESULT

Pilots see conflicts while they plan, before the authority has to point them out.

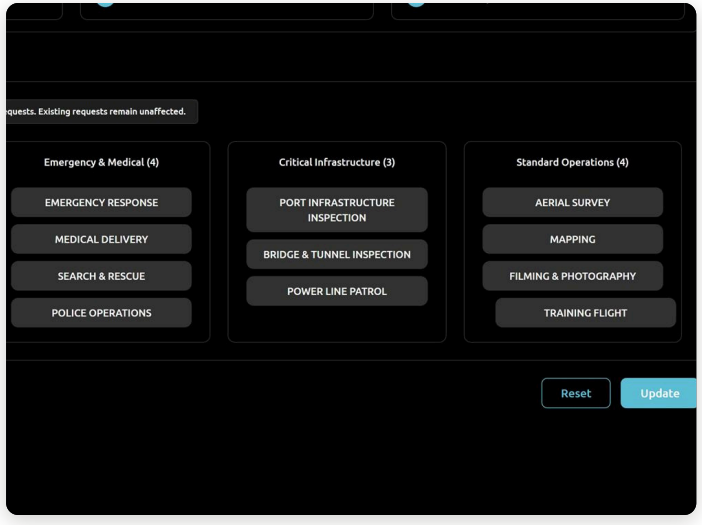
+ More work on the AstraUTM platform

React · TanStack Query · deck.gl



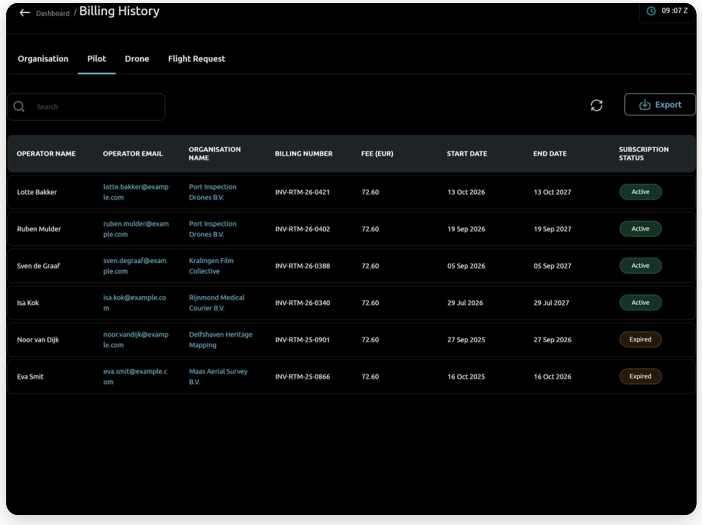
Live COMMS alerts

The unread chat count, refreshed every 5 seconds with a sound on new messages, and a list that opens the conversation for that flight.



Flight priority settings

Drag-and-drop ordering of flight priorities, with the add and edit API behind it.



Billing history

Subscription and registration billing in one table, with active and expired status, behind a feature flag and permission.

Also: 3D NOTAM volumes with deck.gl; the commercial organisation edit page moved to TanStack Query, with 13 test cases; custom fields for users, drones and flights; security fixes that send expired sessions to an unauthorised page across 74 files, plus dependency upgrades and removal of keys from shipped config.

09 Code reviews

120 merge requests · GitLab · SonarQube

I reviewed 120 merge requests that went into the AstraUTM platform, and I run colleagues' changes locally before approving. Some of the issues I raised:

MEMORY

Releasing temporary file links

A file viewer released its object URL on a 10-second timer. I asked for proper clean-up, so links neither leak nor disappear while the file is still open.

TRANSLATIONS

A consistent migration

During the move to Paraglide I flagged old translation hooks, leftover `t()` calls, keys that should be shared, and an altitude label that ignored the feet and metres setting.

QUALITY

Clean merges

Missing alt text on images (an accessibility and SonarQube warning), unused imports, a stray console call in a test, and conflicts left unresolved.

TESTS

Stable selectors

I asked for test IDs instead of text or class selectors in a colleague's tests, and agreed a pattern for dashboard rows: a test ID plus the flight's ID as a data attribute.

SAFETY

Defensive data access

Safe access to nested API responses when saving an incident, and removal of hard-coded strings from user pages.

VERIFICATION

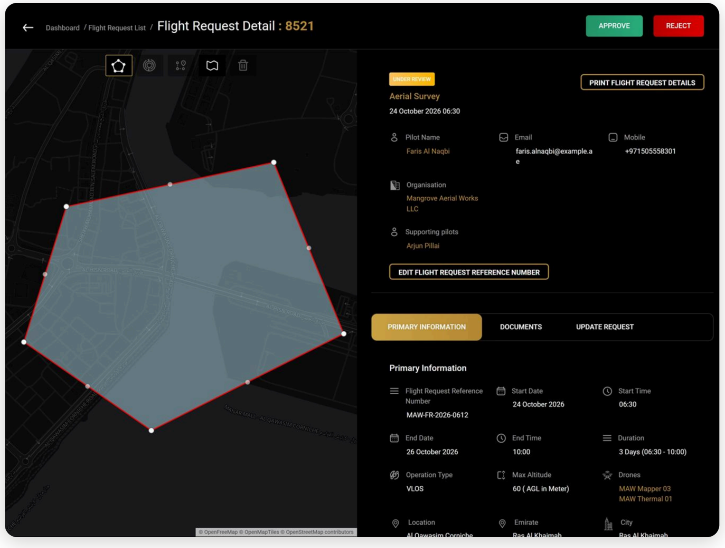
Checked locally

For changes with a wide reach I run the branch, check the dashboard and the other critical pages, and confirm the result in the review.

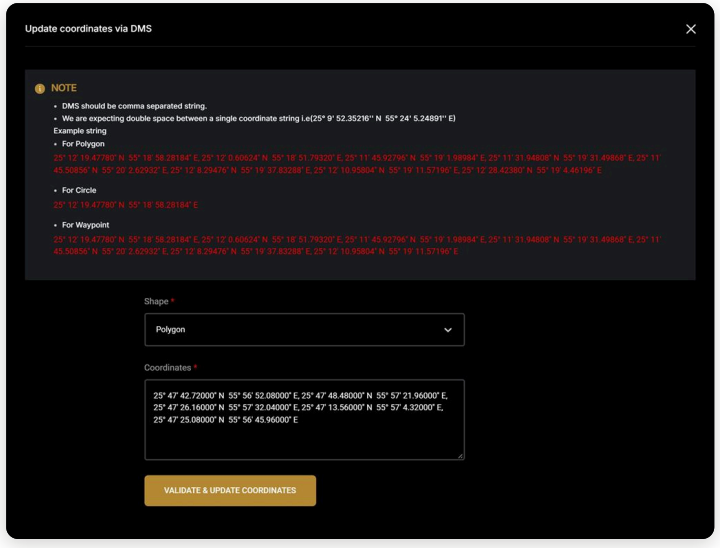
10 The flight-request review page

React · Google Maps · REST APIs

Authority staff decide on every drone flight request from this page. I created it in 2022, extended it for four years, and in 2025 split it into 12 focused components.



A request under review: area, pilot, drone and status.



Amending a request with coordinates in degrees, minutes and seconds.

THE PROBLEM

The page had grown past 2,000 lines in one component, so every change risked breaking another section.

MY PART

- The page itself: map area, pilot, drone, insurance, risk, billing, certificates, documents and comments.
- 12 section components split out of the original file.
- Amendments with coordinates in degrees-minutes-seconds or decimal, converted both ways.
- Bulk download of all attachments, and a payment waiver for staff.

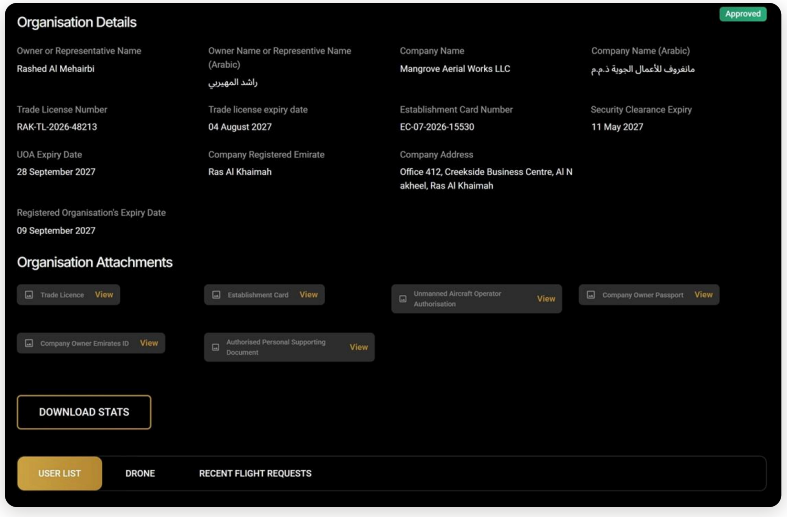
RESULT

Half of the page's code is mine: 2,800 of 5,600 lines, built over 247 commits.

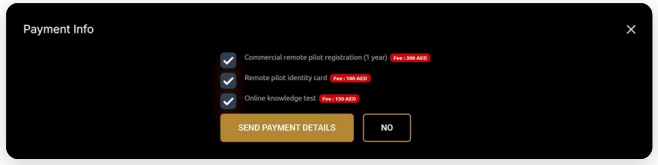
11 Organisation approval and payment requests

React · Reusable components · REST APIs

Commercial operators must be approved, and often pay a registration fee, before they can fly. I built the authority's review screens and one payment component used in four approval flows.



Reviewing an organisation: licence, owner, documents and its users.



The payment request, from the same component in every flow.

```
// one component, four approval flows
<PaymentInfobox
  fetchUrl={`/payments/organisation/${id}`}
  postUrl={"/payments/request"}
  paymentCategory={Payment.Organisation}
/>
```

THE PROBLEM

Users, organisations, drones and flight requests each needed the same "request a fee, or waive it" step during approval.

MY PART

- The organisation review: trade licence, establishment card, owner's ID and documents, with approve, reject and activate.
- A payment component driven by props, reused in all four approval flows.
- Payment references, the rejected-applicant history and the system configuration page.

RESULT

One payment step behaves the same everywhere, and parts of these screens carried over into today's platform.

12 Security fixes and safe navigation

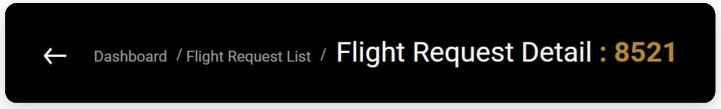
React Snyk SonarQube

In 2025 I worked through security findings in the first-generation app, from how attachments are opened to how every page builds its breadcrumb.

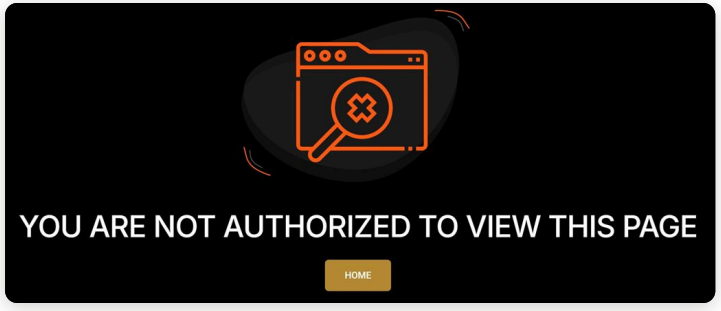
```
// breadcrumbs from a route map instead of
// a 990-line chain of if statements
const routeMatchers = [
  { test: /^\/flight-view-page\//, key: "flightView" },
  { test: /^\/commercialorg\/view\//, key: "orgView" },
  // …
];

const resolveCrumbs = (path) => {
  const match = routeMatchers
    .find((m) => m.test.test(path));
  return match ? breadcrumbMap[match.key] : [];
};

// only known in-app paths can be followed
const safeRoute = (to) => {
  const external = /^(https?:|\/\/|javascript:|data:)/i;
  if (external.test(to)) return "/";
  const known = allowedPrefixes
    .some((p) => to.startsWith(p));
  return known ? to : "/";
};
```



A breadcrumb built from the route map.



Where an expired or unauthorised session lands.

THE PROBLEM

Attachments opened from public links, and the breadcrumb was a 990-line chain of if statements that security scans flagged.

MY PART

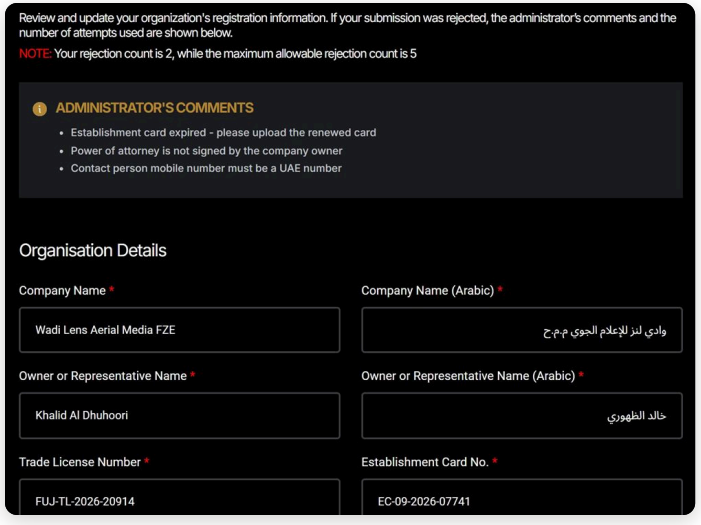
- Attachments fetched through authenticated requests as files, from one button used on 10 screens.
- Shared helpers for safe links and bulk downloads, used across 15+ files.
- Allow-listed IDs in history links, and an unauthorised page for expired sessions.
- The breadcrumb rewritten as a route map with a resolver and a safe-route check.
- SonarQube and Snyk fixes and dependency updates.

RESULT

Attachments open only through authenticated requests, and breadcrumb links can only point inside the app.

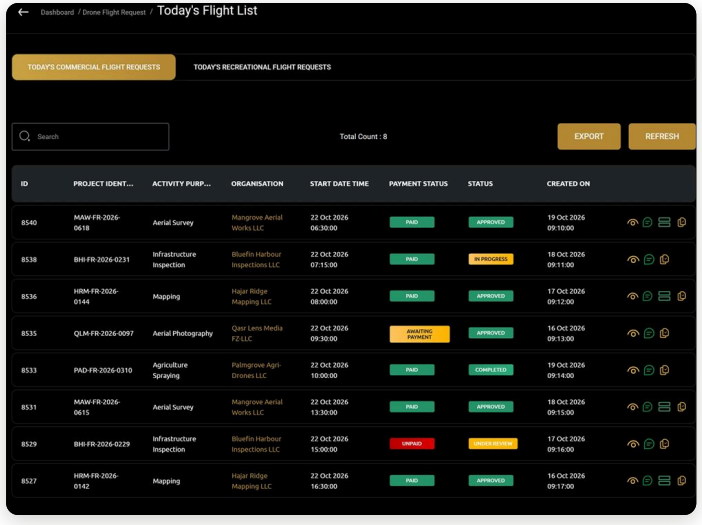
+ More work on SCNS

React · Forms · AG Grid



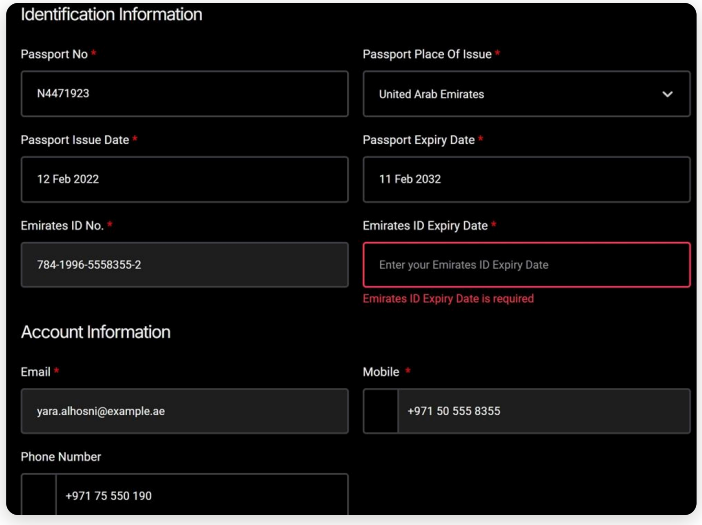
Resubmitting a rejected application

Applicants confirm their email with a one-time code, then correct their details and documents and resubmit, up to the allowed number of attempts.



Today's flights

Today's commercial and recreational flights in tabs, and a popup that lists every flight in a multi-flight request.



Pilot and company sign-up

Sign-up forms for pilots and companies with national ID, passport and trade-licence fields, one-time codes and validation.

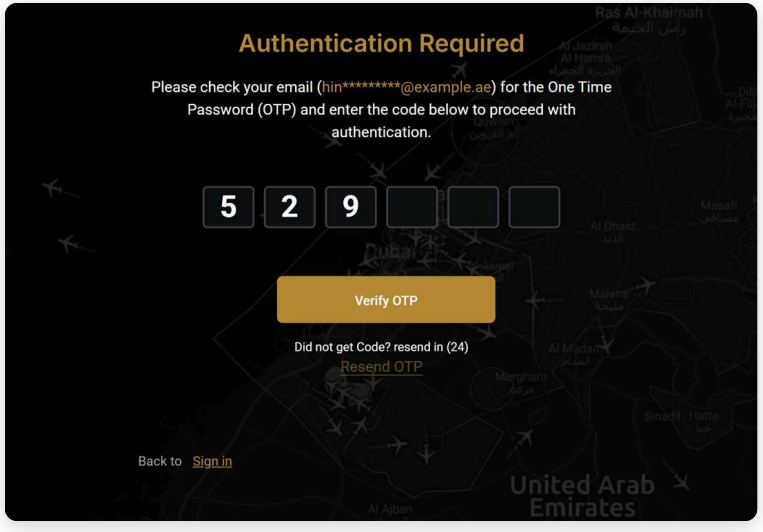
Also: tracker registration with cascading organisation, user and drone selection, still in use on today's platform; area broadcasts that send a message to every drone inside a drawn polygon; the shared AG Grid table, which I introduced and 45 screens now use; and 146 release promotions from development to staging and production.

13

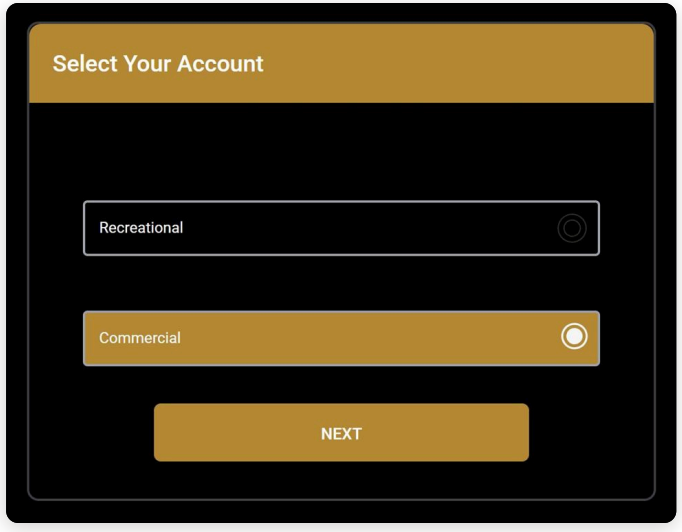
Two-step sign-in for the control-room portal

JavaScript · jQuery · ASP.NET MVC

Staff sign in with a password and a one-time code, and operators sign in with the national digital ID. I built both flows in the authority's ASP.NET portal.



The one-time code: masked email, focus moving box to box, resend countdown.



Choosing a profile when one person holds two.

```
// paste a whole code into the first box
inputs[0].addEventListener("paste", (e) => {
  const code = e.clipboardData.getData("text").replace(/[^a-z0-9]/gi, "");
  e.preventDefault();
  inputs.forEach((box, i) => (box.value = code[i] ?? ""));
  if (code.length === inputs.length) submitOtp(code);
});
```

THE PROBLEM

The portal needed a second factor for staff and national-ID sign-in for operators, in English and Arabic.

MY PART

- A segmented code input: auto-advance, Backspace goes back, a pasted code fills every box and submits.
- A resend countdown and a masked email in the prompt.
- National-ID sign-in that returns users to the page they asked for, and a picker for users with two profiles.
- Messages for cancelled, under-review and failed sign-ins in English and Arabic.
- The server actions behind them, with secure cookies and anti-forgery checks.

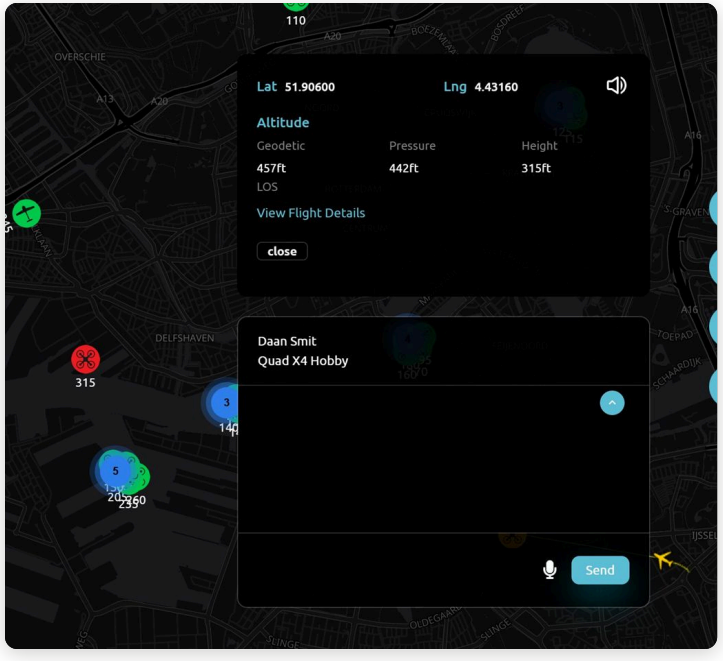
RESULT

Every staff sign-in has a second factor, and operators sign in with the ID they already use.

14 Live traffic alerts in the control room

JavaScript · Google Maps API · MarkerClusterer

Control-room staff watch every live drone and aircraft on one map. In the authority portal I added clustering for busy areas, a repeating alarm for unauthorised drones, pilot positions and fuller altitude details.



An unauthorised drone (red): geodetic and pressure altitude, and its alarm control.

```
// one alarm per unauthorised drone
const alarms = new Map(); // flightId -> timer

function syncAlarms(unauthorised) {
  for (const id of unauthorised) {
    if (alarms.has(id)) continue;
    const timer = setInterval(
      () => siren.play(), 3000);
    alarms.set(id, timer);
  }
  // drones that left the feed go quiet
  for (const [id, timer] of alarms) {
    if (unauthorised.has(id)) continue;
    clearInterval(timer);
    alarms.delete(id);
  }
}

// rebuild clusters at most once a second
const refreshClusters = debounce(
  () => clusterer.repaint(), 1000);
```

THE PROBLEM

With many drones in a small area the map became hard to read, and an unauthorised drone was easy to miss.

MY PART

- Clustering of live tracks, rebuilt at most once a second so the map stays responsive.
- A repeating alarm for each unauthorised drone, with a mute per drone, cleared when the drone leaves the feed.
- Pilot positions drawn next to their drones.
- Geodetic and pressure altitude in each track's information box.
- A line-of-sight request removed from every refresh of the information box, so each update sends fewer requests.

RESULT

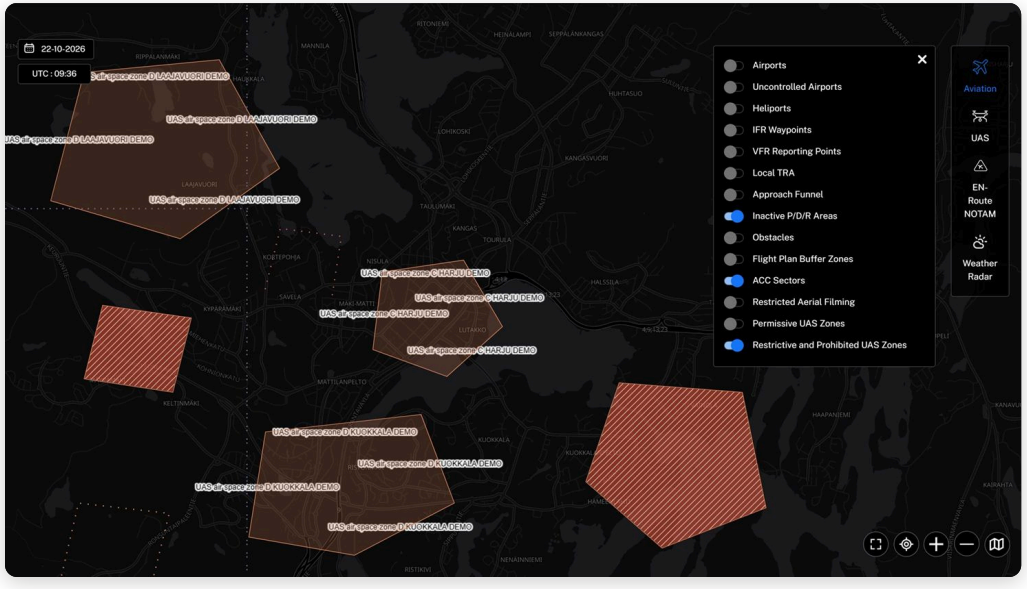
Busy areas stay readable, and an unauthorised drone is heard as soon as it appears on the map.

15

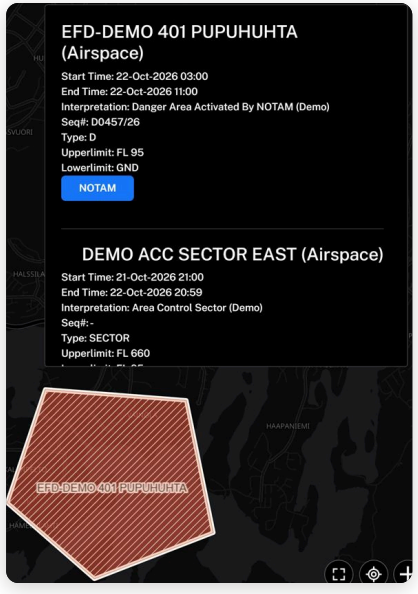
Airspace layers on a public map

Google Maps API · SVG · Turf.js

Fintraffic Sky shows drone pilots in Finland which airspace applies where. I built the map layers that make zone types easy to tell apart.



Hatched danger areas, dashed sector lines and labels inside each zone.



The airspace at a clicked point, with times and limits.

THE PROBLEM

Overlapping zones of different types looked alike on the map, and their names were hard to place.

MY PART

- A hatched fill for danger and prohibited areas, using an SVG pattern inside a custom map overlay.
- Dashed sector boundaries from a repeated SVG symbol.
- Zone names placed inside each border by shrinking the polygon with Turf.js, grouped so labels show at the right zoom.
- A toggle for inactive danger and restricted areas, and an information box with times and limits for any point.

RESULT

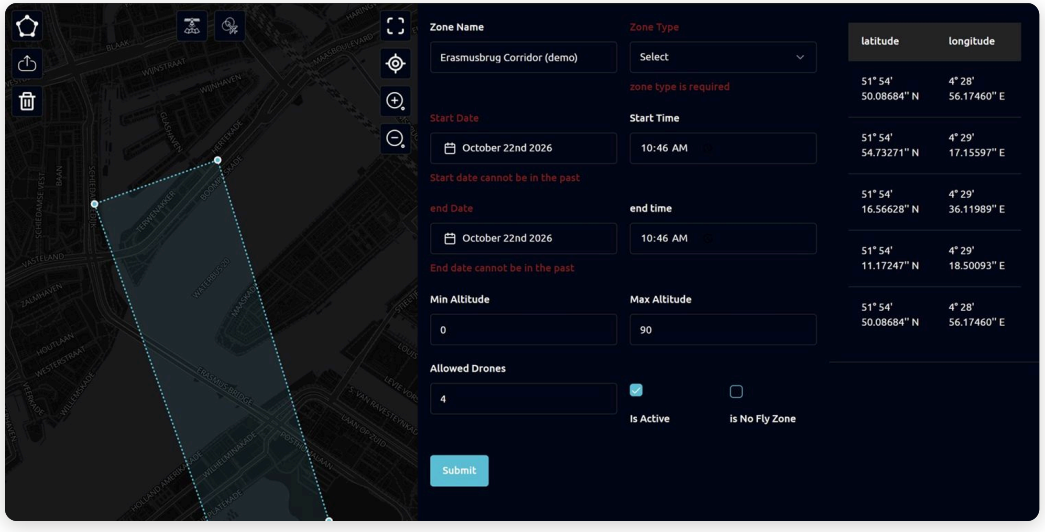
Each zone type has its own look on the map, and the hatched layer was later reused in a second product.

16

Building the platform in TypeScript

In 2024 the team built a TypeScript version of the platform. I wrote 3,100+ lines of it: zone, alert and deconfliction screens and their typed APIs.

TypeScript · TanStack Router · React Hook Form · Zod



A zone drawn on the map: its coordinates in degrees, and the schema's messages for a missing type and past dates.



A request's detail panel with its next actions.

THE STACK

TanStack Router with file-based routes, TanStack Query, React Hook Form with Zod schemas, Tailwind and shadcn components.

MY PART

- Zone and approver-zone forms drawn on the map, with typed zone APIs and Zod schemas.
- Alert thresholds and the deconfliction edit screen, each validated by a schema.
- The calendar view and the request detail panel with altitude and radius sliders and approve or reject.

RESULT

The same patterns, typed APIs and schema-first forms, carried into my TypeScript work on the main platform.